Our-Map — Ghid de Prezentare (Versiunea Light)

Document de familiarizare cu structura HTML și CSS a paginii de prezentare (Landing Page), creat pentru prezentarea la disciplina Tehnologii Web (TW) — branch **light_version**.

1. Ce am modificat față de versiunea anterioară

Scopul acestei versiuni (`light_version`) a fost să simplificăm codul paginii de prezentare astfel încât fiecare linie de HTML și CSS să fie ușor de înțeles, de explicat și să nu mai existe suprapuneri pe ecranele de telefon.

- **HTML curat direct în `index.html` :** Înainte, tot codul HTML al paginii de prezentare era generat din JavaScript (în `landing.js`). Acum structura semantică (`<header>` , `<nav>` , `<main>` , `<section>` , `<footer>`) este scrisă direct și comentată clar în `index.html` .
- **CSS simplificat în `src/css/landing.css` :** Am redus fișierul de la peste 900 de linii la ~380 de linii clare și comentate. Am eliminat efectele complexe de tip `blur` , `radial-gradient` , `backdrop-filter` , `clamp()` și animațiile `@keyframes` .
- **Fără suprapuneri pe telefon:**
  - Am eliminat widget-ul flotant de test `?` (DevLog cu `position: fixed`) care acoperea bara de adresă a mockup-ului pe ecrane înguste.
  - În previzualizarea hărții (`.mockup-map-area`), am înlocuit pinurile poziționate absolut (`position: absolute`) cu un layout normal bazat pe **Flexbox** și **CSS Grid**, astfel încât cardurile să se așeze unul sub altul pe telefon, fără să se atingă sau să se acopere.
- **Organizarea fișierelor pe scopuri:** Am separat fișierele CSS în folderul `src/css/` (`landing.css` pentru pagina de prezentare și `map.css` pentru harta interactivă Leaflet).

2. Structura proiectului (pe scurt)

Fișier	Rol în aplicație
`index.html`	Conține structura HTML5 completă a paginii de prezentare și containerul `#app` pentru hartă.
`src/css/landing.css`	Toate stilurile pentru landing page: culori, navbar, hero, grid-uri și media queries pentru mobil.
`src/css/map.css`	Stilurile separate pentru harta interactivă Leaflet, panoul lateral și modalele de adăugare pin.
`src/landing.js`	Doar logica JS a butoanelor de pe landing page (deschiderea hărții, autentificarea și actualizarea navbar-ului când ești logat).
`src/main.js`	Comută între cele două vederi (`view-landing` și `view-map`) și pornește harta la nevoie.

3. Cum este structurat HTML-ul (`index.html`)

Pagina folosește etichete HTML5 semantice, împărțite logic în 4 zone principale:

1. `<head>`: Include `<meta name="viewport" content="width=device-width, initial-scale=1.0" />` (obligatoriu pentru ca site-ul să se adapteze la lățimea ecranului de telefon) și `<link rel="stylesheet" href="/src/css/landing.css" />`.
2. `<header class="navbar">`: Bara de sus (sticky) care conține logo-ul (`.brand`), meniul cu ancore către secțiuni (`<nav class="nav-links">`) și butoanele de acțiune (`Autentificare` și `Deschide Harta`).
3. `<main>`: Conține cele 4 secțiuni ale paginii:
   - `.hero` — Titlul principal (`<h1>`), subtitlul, butoanele CTA, lista de beneficii și previzualizarea hărții (`.mockup-card`).
   - `#features` — Cele 4 funcționalități principale, fiecare într-un `<article class="feature-card">`.
   - `#how-it-works` — Ghidul în 3 pași (`<article class="step-card">`).
   - `#about` — Bannerul final de îndemn la acțiune (`.cta-box`).
4. `<footer class="footer">`: Logo-ul, linkurile rapide către secțiuni / GitHub și textul de copyright.

4. Cum funcționează CSS-ul (`src/css/landing.css`)

Fișierul CSS este organizat în 10 secțiuni numerotate, ușor de urmărit:

4.1. Variabilele de culoare (`:root`)

La începutul fișierului definim culorile o singură dată în `:root`, apoi le refolosim peste tot cu `var(--nume)`. Dacă vrem să schimbăm tema de culoare, modificăm într-un singur loc.

```
:root {
  --bg-main: #0f172a;      /* Fundal principal albastru închis */
  --bg-card: #1e293b;      /* Fundal pentru carduri */
  --border-color: #334155; /* Bordură subtilă */
  --accent: #e11d48;       /* Roz-coral pentru butoane */
  --text-main: #f8fafc;    /* Text principal alb */
  --text-muted: #cbd5e1;   /* Text secundar gri deschis */
}
```

4.2. Containerul central și Box Model

Clasa `.container` limitează conținutul la maximum `1100px` și îl centrează pe ecran cu `margin: 0 auto`. Prin `box-sizing: border-box`, padding-ul și bordura nu măresc lățimea totală a elementelor, evitând depășirea ecranului.

4.3. Butoane reutilizabile (clase compuse)

Avem o clasă de bază `.btn` (padding, colțuri rotunjite, aliniere cu `inline-flex`) și clase modificatoare: `.btn-primary` (coral), `.btn-secondary` (gri închis cu bordură), `.btn-lg` (butoane mari) și `.btn-sm` (butoane mici).

4.4. Flexbox vs. CSS Grid în pagină

Tehnică	Unde o folosim	De ce am ales-o
Flexbox (`display: flex`)	`.navbar-inner` , `.hero-actions` , `.mockup-topbar` , `.footer-inner`	Ideal pentru elemente pe o singură direcție (rând), cu `justify-content: space-between` și `flex-wrap: wrap` ca să treacă pe rândul următor dacă nu încap.
CSS Grid (`display: grid`)	`.mockup-pins-grid` (2 col.), `.features-grid` (2 col.), `.steps-grid` (3 col.)	Ideal pentru tabele de carduri egale. Pe desktop punem `repeat(2, 1fr)` sau `repeat(3, 1fr)` , iar pe telefon schimbăm într-o singură linie la `1fr` .

4.5. Responsive Design (`@media`)

Folosim două praguri (breakpoints) foarte clare pentru ecrane mici:

```
@media (max-width: 768px) {
  .nav-links { display: none; } /* Simplificăm navbar-ul pe mobil */
  .hero-title { font-size: 30px; } /* Titlu mai mic față de 46px pe desktop */
  .mockup-pins-grid,
  .features-grid,
  .steps-grid {
    grid-template-columns: 1fr; /* Toate cardurile pe 1 singură coloană */
  }
  .footer-inner { flex-direction: column; text-align: center; }
}

@media (max-width: 480px) {
  .hero-actions .btn { width: 100%; } /* Butoane pe toată lățimea pe telefoane mici */
}
```

4.6. Comutarea între Landing Page și Hartă

Elementul `<body>` are clasa `view-landing` (implicit) sau `view-map` (când apăsăm „Deschide Harta”). În CSS, ascundem secțiunea inactivă:

```
body.view-map #landing-page { display: none; }
body.view-landing #map { display: none !important; }
```

5. Cum argumentezi criteriile de evaluare TW

- **Contrast:** Fundal închis (`#0f172a`) cu text alb (`#f8fafc`) și butoane coral (`#e11d48`) — lizibilitate foarte bună.
- **Aliniere:** Tot conținutul respectă axa centrală dată de `.container` (`max-width: 1100px`).
- **Proximitate:** Informațiile înrudite (iconiță + titlu + descriere) sunt grupate în carduri distincte (`.feature-card` , `.step-card`).
- **Repetiție & Consistență:** Aceleași raze de rotunjire (`10px–16px`), aceeași culoare de bordură (`#334155`) și aceleași clase de butoane pe toată pagina.
- **Arhitectura informației:** Flux logic de sus în jos: ce face aplicația (Hero) → ce funcții are (Features) → cum se folosește (3 pași) → acțiune (CTA) → subsol (Footer).

6. Întrebări posibile de la profesoară și răspunsuri rapide

1. Ce face `box-sizing: border-box`?

Face ca `padding`-ul și `border`-ul să fie incluse în lățimea (`width`) declarată a unui element, nu adăugate pe lângă ea. Astfel cardurile nu ies din container pe telefon.

2. De ce ai folosit atât Flexbox, cât și CSS Grid?

Flexbox este pentru așezări pe o singură direcție (bara de navigație, grupul de butoane, footer-ul), iar CSS Grid este pentru așezări pe coloane egale (grid-ul 2x2 de funcționalități și cele 3 coloane de pași).

3. Cum ai rezolvat suprapunerile de pe telefon?

Am scos butonul flotant de test cu `position: fixed` și am înlocuit pinurile poziționate absolut (`position: absolute`) din previzualizarea hărții cu un `CSS Grid`. Pe ecrane sub `768px`, toate grid-urile devin o singură coloană (`1fr`), iar butoanele folosesc `flex-wrap: wrap`.

4. De ce ai păstrat și JavaScript în proiect?

Pagina de prezentare este construită în HTML și CSS nativ direct în `index.html` și `src/css/landing.css`. JavaScript-ul este separat în `src/` doar pentru harta interactivă Leaflet și conectarea cu baza de date Firebase.